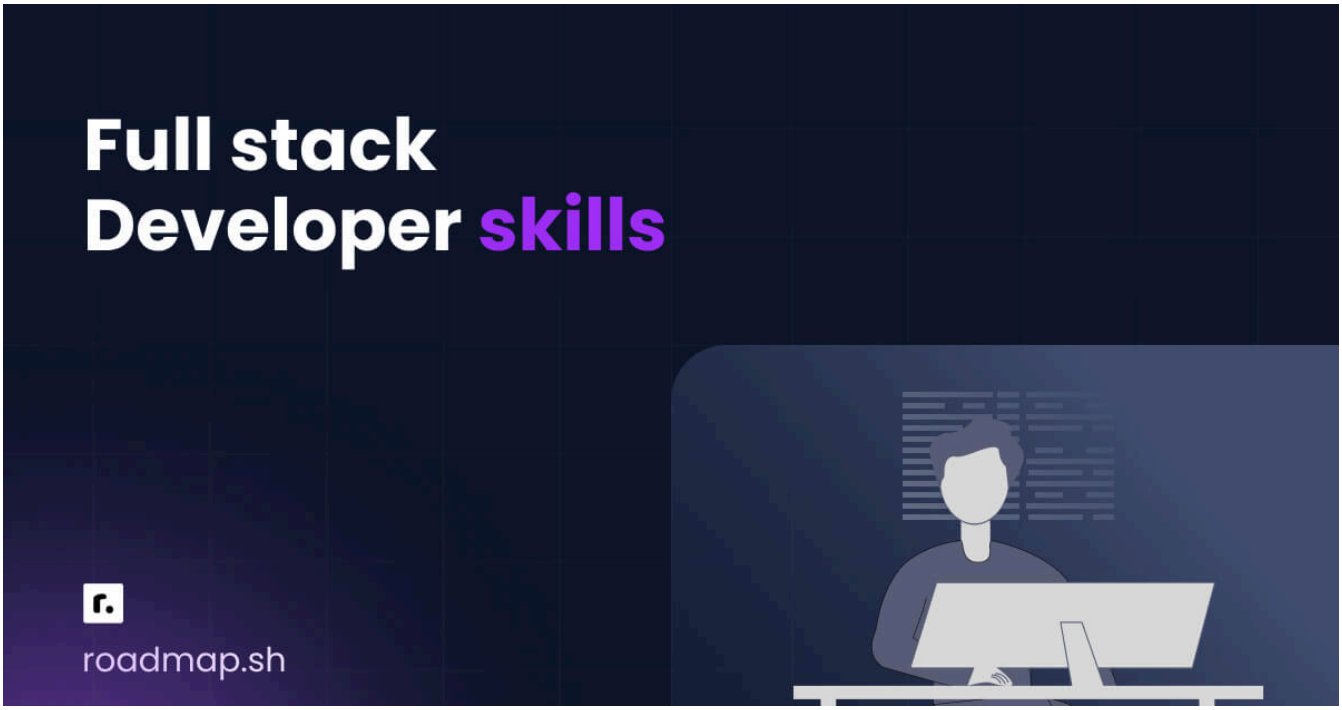
NEW Announcing our new AI engineering course: **How LLMs Work**

Roadmaps AI Tutor Upgrade Login Sign Up

8 In-Demand Full Stack Developer Skills to Master



Fernando Doglio

Prefer us on Google

It should be no surprise to anyone that given the fast pace of the web
pecially full stack
nuously updating their

expertise that allows
veb applications.
However, this mix of skills is a double-edged sword because it makes

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Customise Reject All Accept All

staying up-to-date harder for them. In this article, we will explore the 8 essential full stack developer skills that you need to master and thrive in your career.

In this Article

Understanding full-stack development
1. Basic understanding of HTTP
2. HTML/CSS/JavaScript
3. Writing Modern JavaScript
4. At least One Frontend Framework (React, Vue, Angular)
5. Backend Programming Language
6. Consuming and Creating RESTful APIs
7. Databases
8. Hosting and Infrastructure

Remember that you have more details about the different technologies and topics to cover in our **full stack developer roadmap**.

Understanding full-stack development

Full stack development involves both frontend and backend work. It means
website or application
ing the server, database,
s (backend).

working with both
and maintaining
infrastructure, and ensuring that their applications are scalable and secure.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

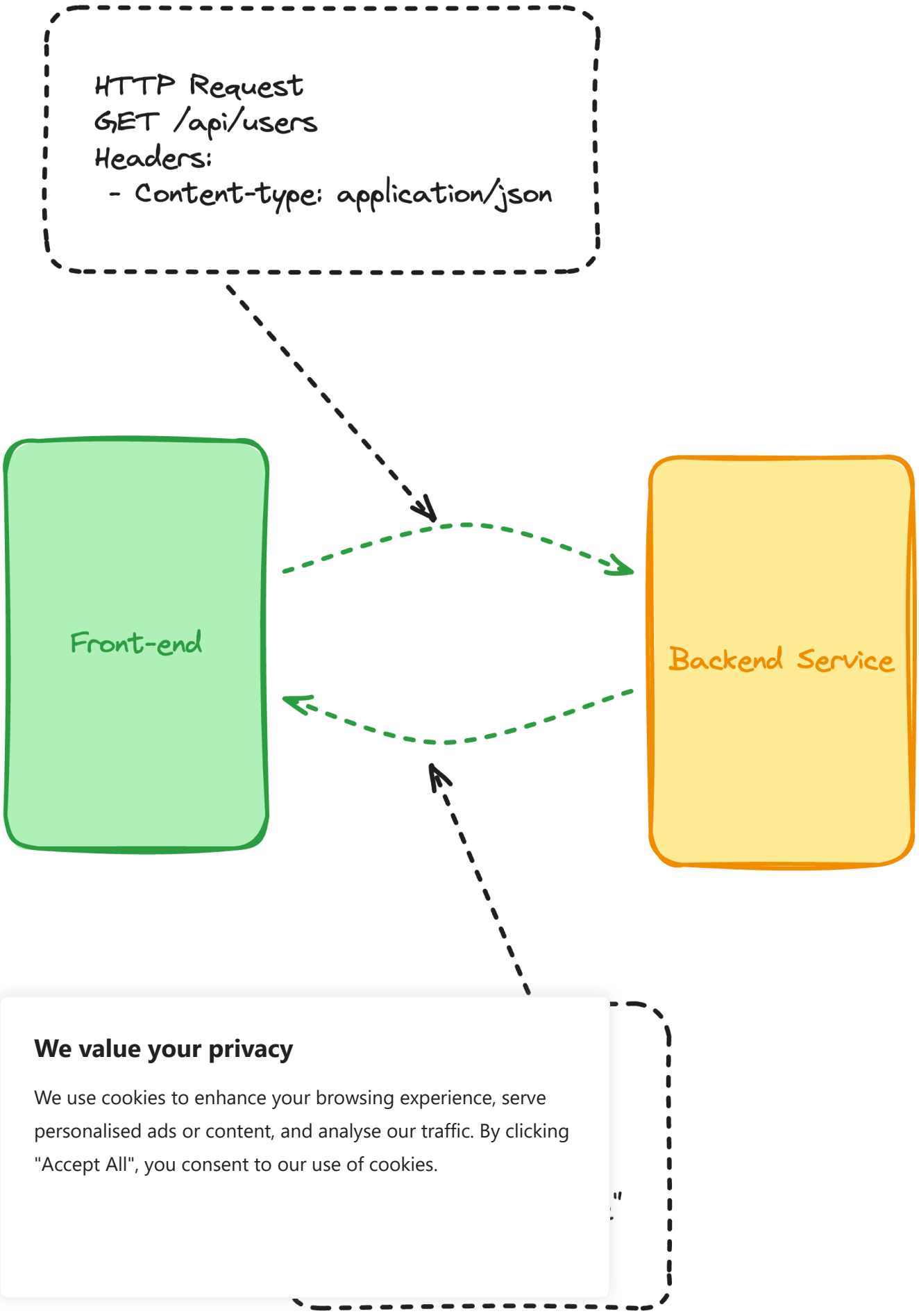
In other words, if they need to, full-stack developers can be the equivalent of a one-man army. Is this an ideal situation? No, it isn't. In fact, a better way to look at a full stack developer is to think of them as a wildcard that can be placed in any team, and they'll consistently add value to it, with minimum ramp-up time needed.

With that foundation in mind, let's dive into the specific skills you need to master.

1. Basic understanding of HTTP

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Let's start with the basics: understanding HTTP. HTTP, or HyperText Transfer Protocol, is the foundation of any interaction on the web. It's what allows clients (like your web browser) and servers to communicate with each other. As a full-stack developer, having a solid grasp of HTTP is crucial for building, debugging, and maintaining web applications. It's your bread and butter, and you'll need to internalize it if you hope to become great at your job.

Why HTTP Matters for Full Stack Developers?

Without HTTP, the internet as we know it wouldn't function, it's that simple. For a full-stack developer, understanding HTTP means you can effectively manage how your frontend communicates with your backend.

Key Concepts to understand in HTTP

- **HTTP Methods**: These are the actions that can be performed on a resource. The most common methods are GET (to retrieve data), POST (to send data to the server), PUT (to update data), and DELETE (to remove data). Each method has a specific purpose and knowing when to use each one is essential for building efficient APIs. You can read the **full list of verbs here**.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

es from the server to the
uest was successful, if
ed. Common status codes
al Server Error), and 403
in debugging and

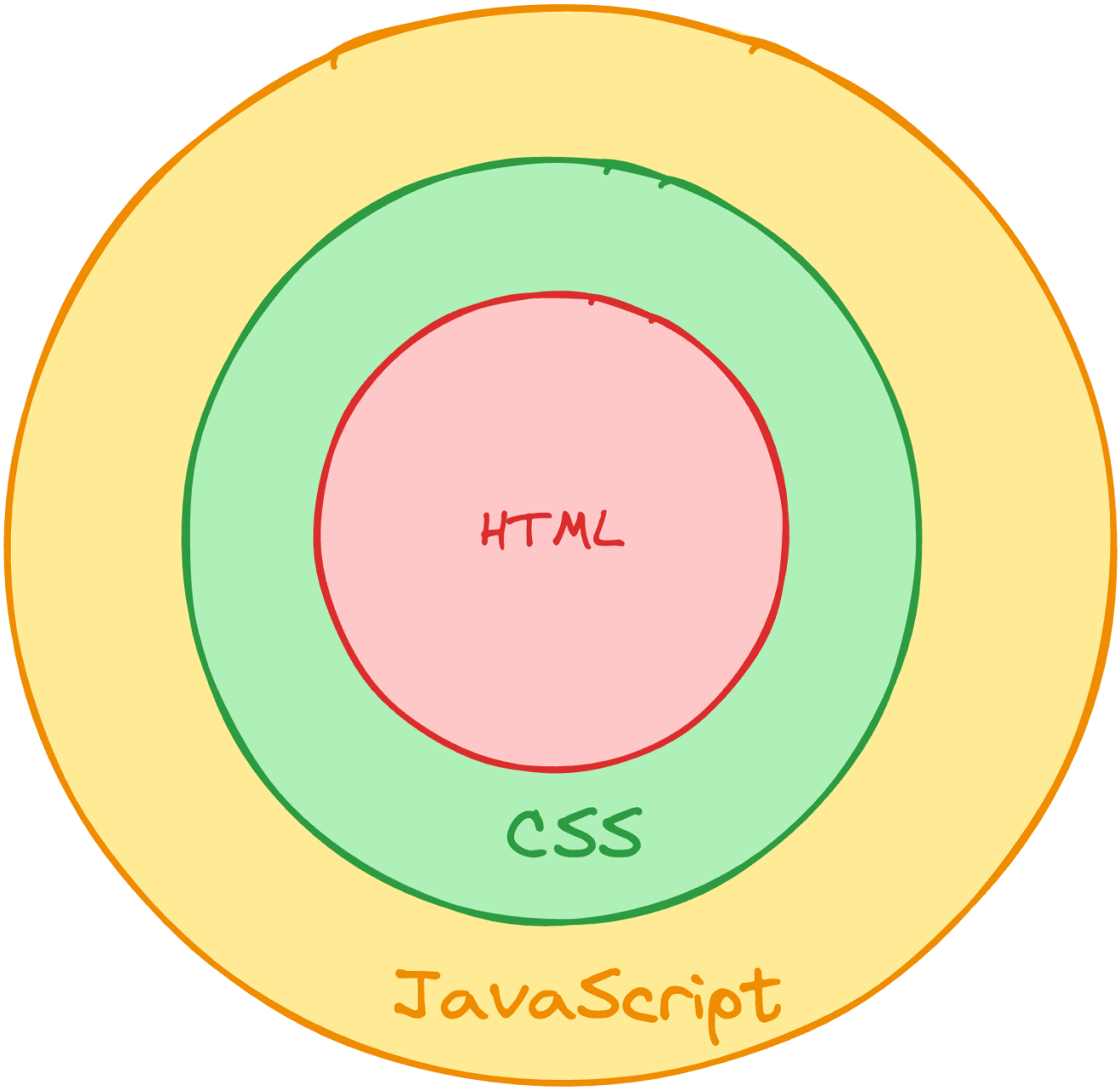
improving user experience. Here's the **full list of status codes** in case you're wondering.

- **Headers**: HTTP headers are key-value pairs sent between the client and server. They carry essential information like content type, authorization information (in some cases even credentials), and cache control. Familiarity with headers allows you to manage security, content negotiation, and request-response flow more effectively. While you can create your own, here's a list of the **standard set of HTTP headers**.

2. HTML/CSS/JavaScript

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



For full-stack developers, a solid grasp of HTML, CSS, and JavaScript is

ks of the visual side of

unctional user interface.

ing these three should be

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

HTML forms the backbone of any web page, providing structure and content. It defines elements like headings, paragraphs, links, images, and multimedia components.

Key Features:

- **Semantic Elements**: Elements like `<header>`, `<footer>`, `<article>`, and `<section>` enhance accessibility and SEO. Properly using these elements makes your content more understandable for users, search engines, and accessibility devices like screen readers.
- **Foundational**: Understanding HTML is crucial for laying the foundation of any web application, making it an essential skill for full stack developers.

CSS (Cascading Style Sheets)

CSS is responsible for the visual presentation of a web page. It controls layout, colors, fonts, and overall style, making your web pages visually appealing.

Key Features:

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

rid Layout, animations,
esigns with minimal

nd Grid are vital for
ces, from desktops to

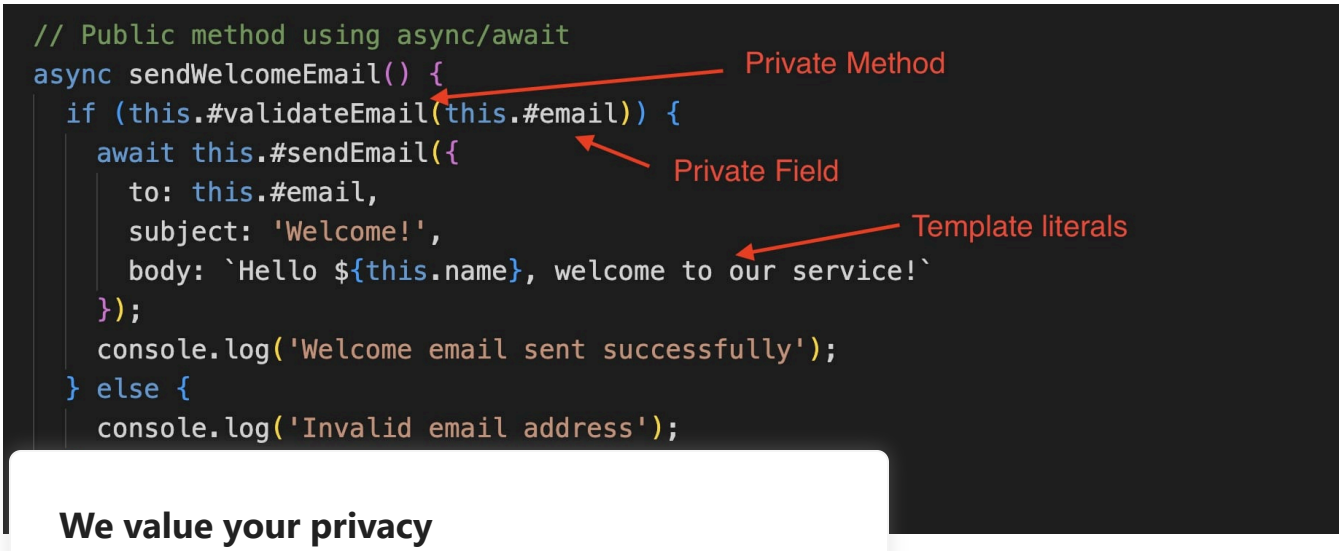
JavaScript

JavaScript adds interactivity and dynamic behavior to web pages. In other words, it makes the webpage come to life.

Key Features:

- **Modern JavaScript**: Features like arrow functions, template literals, destructuring, modules, and promises make your code more concise and readable.
- **Interactive Elements**: JavaScript makes your web pages interactive, responding to user actions in real time.

3. Writing Modern JavaScript



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

evelopment, bringing not
lopment more efficient
g how to stay up-to-date
t also involves

technologies such as: npm, build-tools, and bundlers that streamline the development process.

What is “modern JavaScript”?

The term “Modern JavaScript” can be a bit ambiguous, considering how every new feature or syntax update will directly affect your experience as a developer. Some of those “key modern features” are:

- **Arrow Functions**: Offer a shorthand for writing functions and lexical scoping of the this keyword.
- **Template Literals**: Provide an easier way to create strings with embedded expressions.
- **Destructuring**: Simplifies the extraction of values from arrays or properties from objects.
- **Modules**: Enable better code organization and reuse.
- **Promises and Async/Await**: Facilitate asynchronous programming by providing cleaner and more intuitive ways to handle async operations.

npm (Node Package Manager)

l for managing

u to easily install, update,

up-to-date with the

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Build tools and bundlers to automate many repetitive tasks, optimize performance, and ensure code is production-ready. Let's take a look at some key tools:

- **Webpack**: **Webpack** is a robust module bundler that processes and bundles JavaScript files along with assets like images and stylesheets. It has an ecosystem of plugins and loaders, making it highly configurable and suitable for complex applications.
- **Parcel 2**: **Parcel** simplifies the bundling process with zero configuration out of the box. It automatically handles code splitting hot module replacement and supports a variety of file types.
- **Vite**: **Vite** is designed for speed, leveraging native ES modules to deliver lightning-fast development builds and optimized production builds using Rollup. It's particularly ideal for modern frameworks like Vue, React, and Svelte. Vite's configuration is straightforward, and it supports an extensive plugin system for additional functionality.

4. At least One Frontend Framework (React, Vue, Angular)

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



Although HTML, CSS, and JavaScript form the core of front-end development, there are many frameworks and libraries that can significantly enhance your development workflow. Among the most popular front-end frameworks and libraries are React, Vue, and Angular.

Mind you, there are quite a lot of other options out there. However, React, Vue, and Angular remain the primary choices for most developers due to their widespread use and strong community support.

React

cript library for building

ons.

ture is built around

styles, and behaviors.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

This modular approach promotes reusability and maintainability, making it easier to manage complex applications.

- **Virtual DOM**: React uses a virtual DOM to optimize performance. This technique reduces the need for direct DOM manipulation, which translates into faster updates and a smoother user experience.
- **Large Ecosystem and Community**: With a large ecosystem of libraries and tools, plus a strong community, React provides lots of resources for problem-solving and third-party integrations.

Angular

Developed and maintained by Google, Angular is a full framework for building dynamic web applications

- **Full-Featured Framework**: Angular offers a “complete” solution with built-in support for routing, state management, form handling, and HTTP requests. This approach minimizes the need for additional third-party libraries.
- **Two-Way Data Binding**: Angular’s two-way data binding simplifies the synchronization between the model and the view (the data and its
- **upport**: Angular has
- nmunity, providing a
- ooting.

We value your privacy

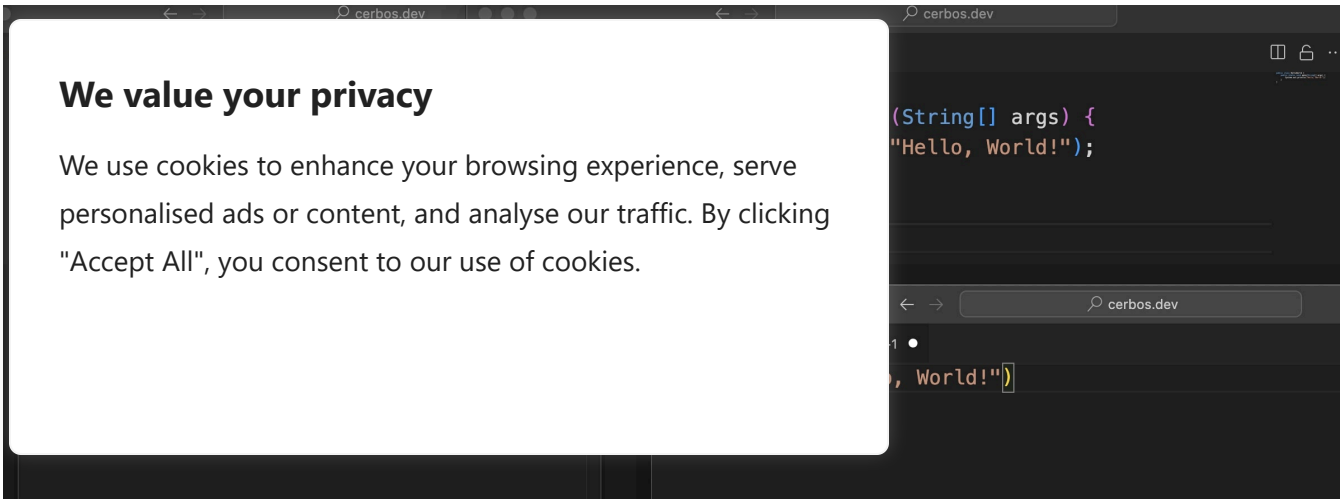
We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Vue.js, created by Evan You after working for Google, is known for its progressive framework design, making it easy to integrate into projects incrementally.

- **Progressive Framework**: Vue is designed to be incrementally adoptable, allowing developers to start with a small part of their application and gradually integrate more features as needed.
- **Simplicity and Ease of Learning**: Vue's straightforward syntax and design make it easier for new developers to learn and start building applications quickly.
- **Flexibility**: Vue offers extensive customization options and can be easily integrated with other projects or libraries. It combines a simple core library with advanced features suitable for larger applications.

Each of these frameworks has its own set of strengths and use cases. The right choice depends on your specific project requirements and personal preference.

5. Backend Programming Language



Choosing the right backend programming language is crucial for building robust and efficient server-side applications. Here are some of the most popular languages used in backend development, each with its own unique features and benefits. You can read this detailed guide for more options and a complete description of each one: **The best backend languages to master**.

JavaScript (Node.js): JavaScript, primarily known for front-end development, extends to the backend through Node.js. Node.js allows for server-side scripting, enabling full stack developers to use the same language for both client-side and server-side development. When it comes to picking a single language for full stack development, JavaScript definitely takes the lead.

Python: Python is well known for its simplicity and readability, making it a favorite among developers. Its vast ecosystem of libraries and frameworks, such as Django and Flask, streamlines backend development and makes full stack web development very straightforward. Python's versatility and ease of use make it suitable for both beginners and experienced developers.

Java: Java is a highly popular, class-based, object-oriented programming l ntation dependencies as l erprise applications. l ance its capabilities.

l natural to read and easy t framework, simplifies the process of building web applications by providing default structures for

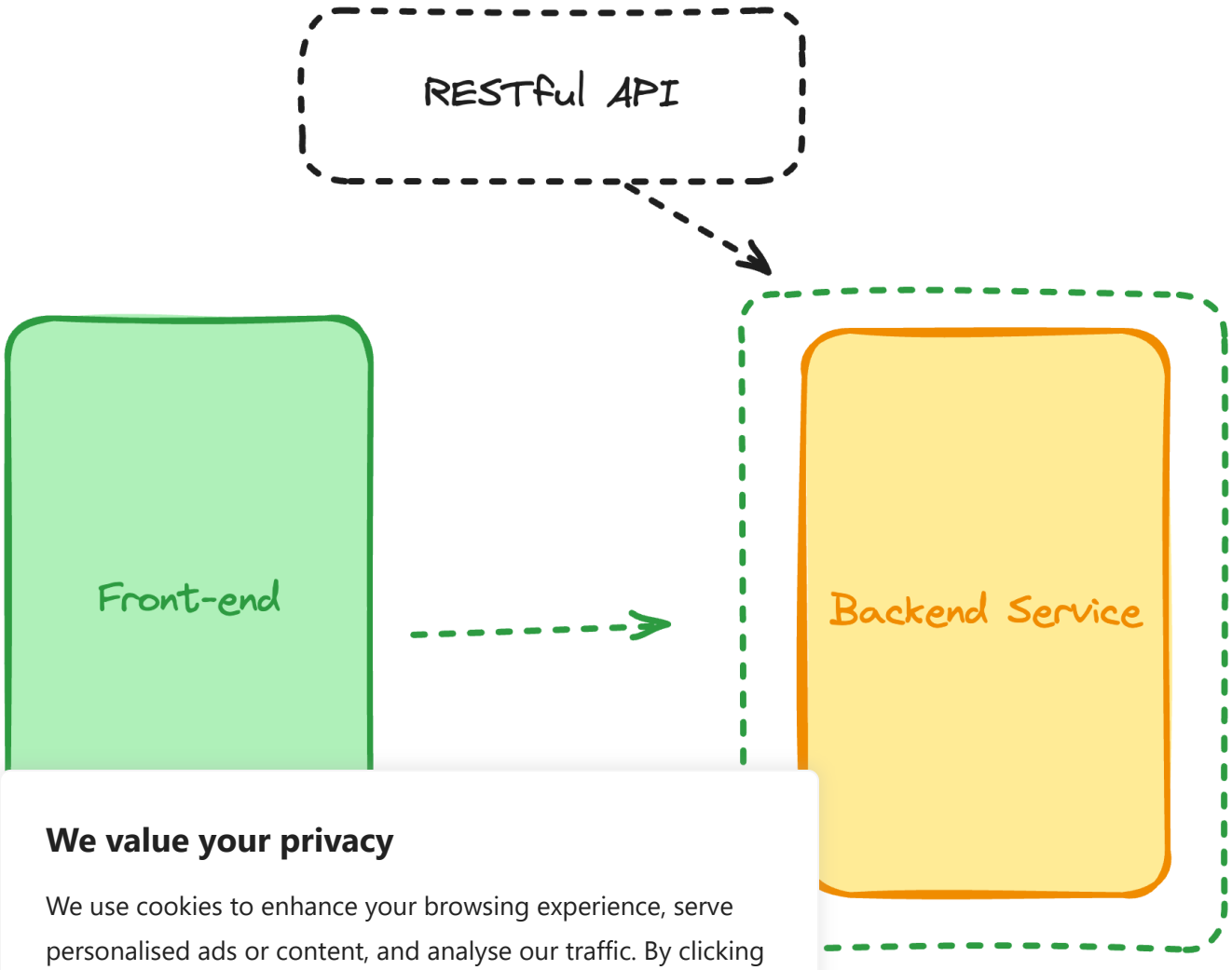
We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

a database, a web service, and web pages. Its convention-over-configuration approach makes development faster and easier.

There are many more options, and as a full stack developer, you'll need to understand the type of technologies you feel more comfortable with, before choosing the language for your backend.

6. Consuming and Creating RESTful APIs



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

s is a fundamental skill

pplications. APIs

(Application Programming Interfaces) allow different software systems to

communicate with each other, enabling your front-end to interact seamlessly with server-side logic and data.

Consuming RESTful APIs

Consuming APIs involves making HTTP requests from your front-end application to retrieve or send data to the server. Here are the key concepts:

- **HTTP Methods**: Understanding the different HTTP methods is crucial. We've already mentioned them above when describing HTTP itself, so make sure to check out the links listed there for more details.
- **Endpoints and Routes**: An API endpoint is a specific path where your API can be accessed by a client. Each endpoint corresponds to a route in your back-end server, handling specific requests. In the case of RESTful APIs, these routes refer to resources in your system.
- **Request and Response**: When consuming an API, the front-end sends a request to the server and receives a response. The request typically includes headers (metadata about the request) and a body (data sent to the server), while the response contains the status code, headers, and data.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

native fetch API or

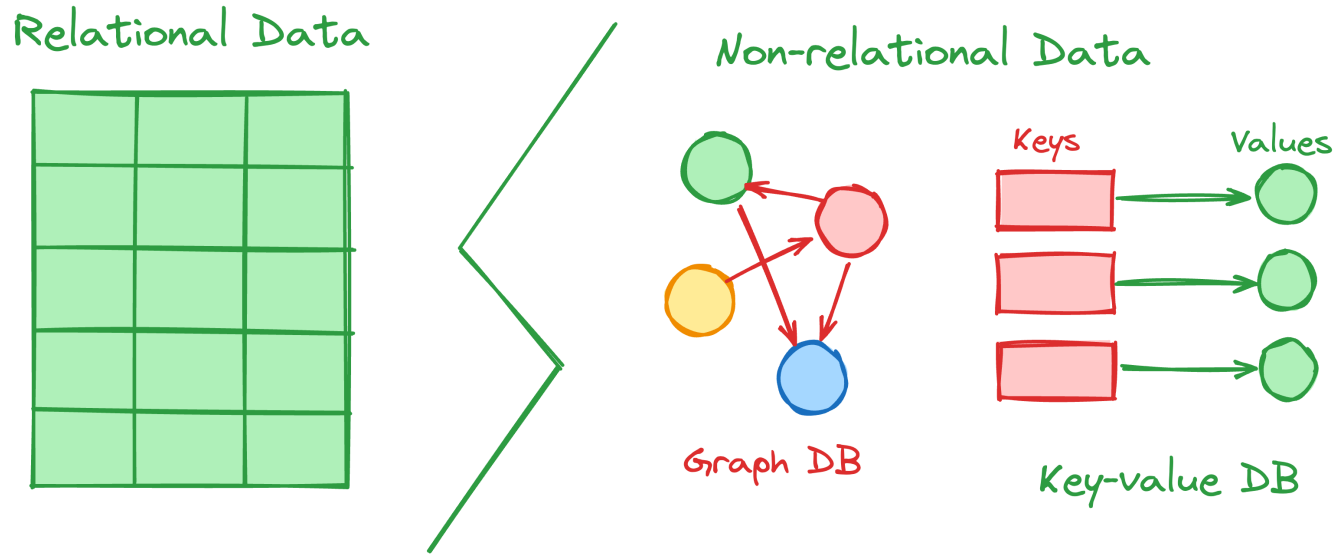
s and handling requests.

Here's how to get started:

- **Define Routes**: In your back-end framework (like Express for Node.js or Django for Python), define routes that correspond to different endpoints. Each route should handle a specific HTTP method.
- **Handle Requests and Responses**: For each route, write logic to handle incoming requests and send appropriate responses. Use status codes to indicate the outcome of the request (e.g., 200 for success, 404 for not found, 500 for server errors).
- **Middleware**: Middleware functions in frameworks like Express can process requests before they reach the endpoint handlers. They're useful for tasks like authentication, logging, and data validation.
- **Database Integration**: Often, your API will interact with a database. Use an ORM (Object-Relational Mapping) tool like Sequelize for SQL databases or Mongoose for MongoDB to manage database operations.
- **REST Principles**: Ensure your API follows REST principles, such as statelessness (each request is independent), resource-based URLs (use nouns for endpoints), and appropriate use of HTTP methods. You can **read here** the full details about REST from the paper that defined the concept.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



Understanding both relational and non-relational databases is crucial for managing and storing data effectively in your applications. Each type of database has its own strengths and use cases, and knowing when to use each is essential for building robust and scalable applications.

Relational Databases

Relational databases store data in structured tables with predefined schemas. They use SQL (Structured Query Language) for querying and managing data. Here are some key relational databases:

- **MySQL**: MySQL is one of the most popular open-source relational databases. It's known for its ease of use (a common choice for new

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

ource relational database
ompliance with SQL
ctions, and extensibility,
ions.

- **SQLite**: SQLite is a lightweight, disk-based database that's easy to set up and use. It's often used in embedded systems and applications that require a simple, self-contained database engine. You're not going to see big systems using SQLite for its main database, but it's very common in small projects.

Non-Relational Databases

Non-relational databases, also known as NoSQL databases, store data in various formats such as documents, key-value pairs, graphs, or wide-column stores. They are designed to handle large volumes of unstructured or semi-structured data. Here are some popular non-relational databases:

- **MongoDB**: MongoDB is a document-oriented database that stores data in JSON-like documents. It's highly flexible and scalable, making it ideal for applications that require quick iterations and schema flexibility.
- **Redis**: Redis is an in-memory key-value store known for its high performance and speed. It's commonly used for caching, real-time analytics, and as a message broker.
- **Cassandra**: Cassandra is a distributed, wide-column store designed for ommodity servers calable and suitable for performance.

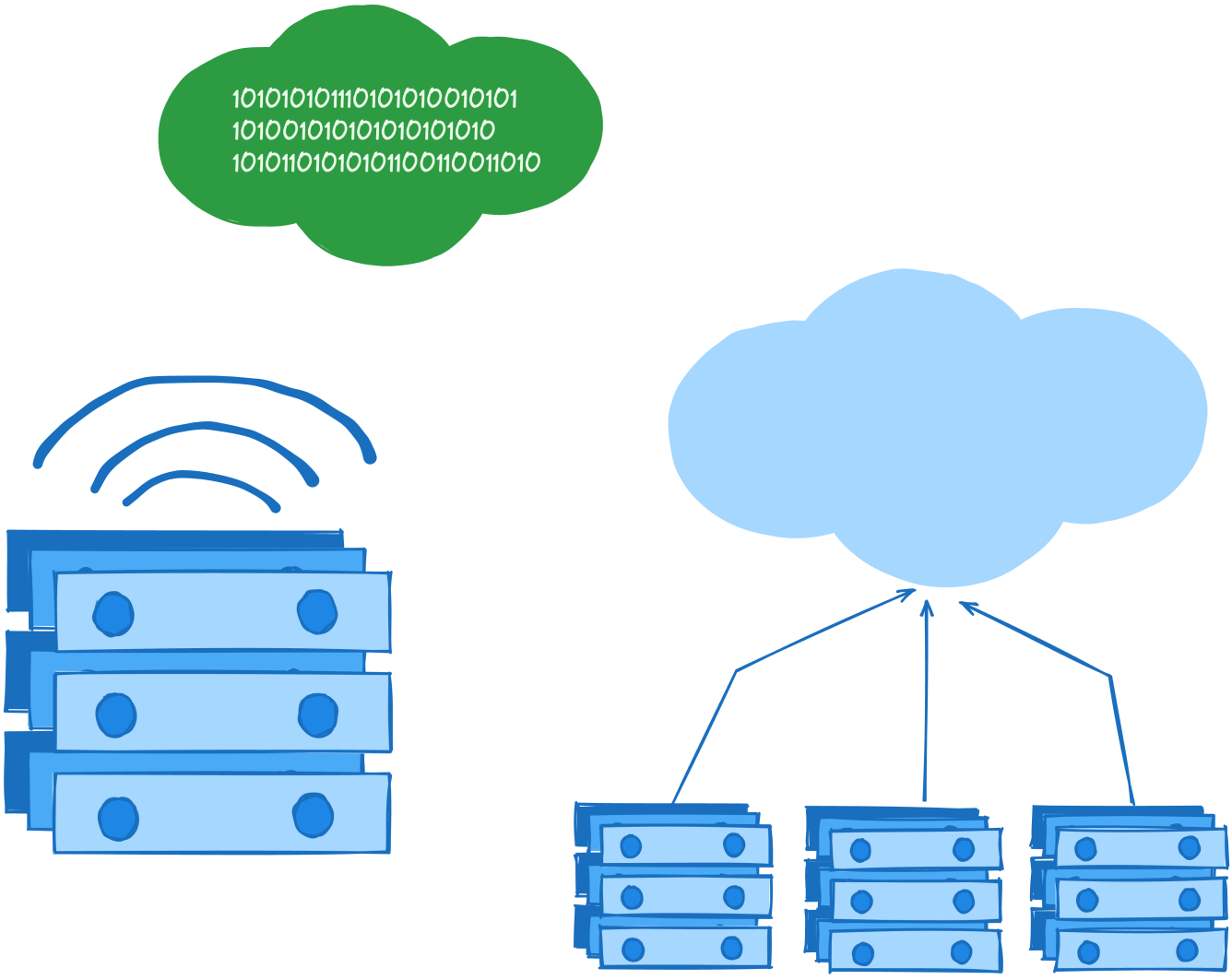
We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

e is not one single option particular needs and the features of each database. When having to decide on a specific database

option, try to understand their capabilities and try to match them to your particular context.

8. Hosting and Infrastructure



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

nd infrastructure is
pplications. This involves
et up servers, and how to
tly. Here's a breakdown

Cloud Hosting Providers

Cloud hosting providers offer scalable and flexible solutions for hosting web applications. They provide various services including virtual machines, storage, databases, and more. Some of the most popular providers are:

- **Amazon Web Services (AWS)**: AWS offers a comprehensive suite of cloud services, including EC2 for virtual servers, S3 for storage, and RDS for managed databases. Its scalability and range of services make it a go-to choice for many developers.
- **Google Cloud Platform (GCP)**: GCP provides robust cloud computing services, including Compute Engine for virtual machines, Cloud Storage, and Cloud SQL for managed databases.
- **Microsoft Azure**: Azure offers a wide range of cloud services similar to AWS and GCP. It includes Azure Virtual Machines, Blob Storage, and Azure SQL Database. Azure integrates well with Microsoft's other products, making it ideal for enterprise applications.

Web Hosting Services

For simpler or smaller-scale projects, traditional web hosting services might be more than enough. These services typically offer easy setup and management for hosting websites and web applications:

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

aS) that makes it easy to
ports several
ghtforward way to deploy
y.

- **Netlify**: Netlify specializes in hosting static websites and front-end applications. It provides continuous deployment from Git repositories, built-in SSL, and a global CDN, making it a popular choice for modern web development workflows.
- **Vercel**: Vercel is optimized for Next.js. It offers seamless deployment, serverless functions, and a global CDN, ensuring fast and reliable performance for web applications.

Containers and Orchestration

Containers allow developers to package applications with all their dependencies, ensuring consistency across different environments (this simplifies deployment processes by ensuring your app always gets deployed into the same system). Orchestration tools manage and scale these containers:

- **Docker**: Docker is a platform that allows developers to create, deploy, and run applications in containers. Containers are lightweight and portable, making it easier to manage application dependencies and environments.
- **Kubernetes**: Kubernetes is an open-source orchestration tool for

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

It automates
ation containers,
applications.

If you just don't want to think about the server, at all (configuration, specs, resource requirements, etc), then you probably want a serverless architecture.

Serverless architecture allows developers to build and run applications without managing server infrastructure (the servers are there, you just don't interact directly with them). Key benefits include automatic scaling and pay-per-use billing.

- **AWS Lambda**: AWS Lambda lets you run code without provisioning or managing servers. It executes code in response to events and automatically scales based on the number of requests, making it ideal for microservices and real-time data processing.
- **Google Cloud Functions**: Google Cloud Functions is a serverless execution environment for building and connecting cloud services. It allows you to write simple, single-purpose functions that are triggered by events.
- **Azure Functions**: Azure Functions provides a serverless compute service that enables you to run event-triggered code. It integrates with other Azure services, offering a seamless development and

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

mputing infrastructure

er than physical hardware

is is a great option if you're looking to version your infrastructure configuration.

- **Terraform**: Terraform is an IaC tool that allows you to define and provision data center infrastructure using a high-level configuration language. It supports multiple cloud providers, making it a versatile choice for managing infrastructure. While not fully open-source, Terraform offers a paid version called Terraform Cloud. If you're looking for a fully open-source version, you can look into OpenTofu, which is an open-source fork from Terraform under the Mozilla Public License 2.0.
- **Ansible**: Ansible is an open-source automation tool for configuration management, application deployment, and task automation. It uses YAML and doesn't require agent software on the target nodes, making it easy to use and manage.

##Conclusion Keeping up-to-date with the full set of skills any full stack web developer will need for their job is not easy, given how varied their work might be. That said, the 8 full stack developer skills listed here are more than enough to get you started, and you can keep adding more as you see the need for them.

Remember that if you want more details about the roadmap to become a ce here: **Full stack**

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Full stack vs Back end: What are the differences?

Full Stack Developer or Software Engineer – Which Way to Go?

Join the Community

roadmap.sh is among the **top 10 most starred projects on GitHub** and is visited by hundreds of thousands of developers every month.

Top 10 out of 28M!

371K

GitHub Stars

Star us on GitHub

Help us reach #1

+90k every month

+3.2M

Registered Users

Register yourself

Commit to your growth

+2k every month

52K

Discord Members

Join on Discord

Join the community

Roadmaps Guides FAQs YouTube

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

THENEWSTACK

The top DevOps resource for Kubernetes, cloud-native computing, and large-scale development and deployment.

DevOps · Kubernetes · Cloud-Native

Cookie Settings

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.